Listing 19-14

TimeCardTransaction.h

```
#ifndef TIMECARDTRANSACTION_H
#define TIMECARDTRANSACTION_H

#include "Transaction.h"

class TimeCardTransaction : public Transaction
{
 public:
  virtual ~TimeCardTransaction();
  TimeCardTransaction(long date, double hours, int empid);

  virtual void Execute();

 private:
  int itsEmpid;
  long itsDate;
  double itsHours;
};
#endif
```

Listing 19-15

TimeCardTransaction.cpp

```
#include "TimeCardTransaction.h"
#include "Employee.h"
#include "PayrollDatabase.h"
#include "HourlyClassification.h"
#include "TimeCard.h"

extern PayrollDatabase GpayrollDatabase;

TimeCardTransaction::~TimeCardTransaction()
{
}

TimeCardTransaction::TimeCardTransaction(long date,
                                         double hours,
                                         int empid)
  : itsDate(date)
  , itsHours(hours)
  , itsEmpid(empid)
{
}

void TimeCardTransaction::Execute()
{
  Employee* e = GpayrollDatabase.GetEmployee(itsEmpid);
  if (e){
    PaymentClassification* pc = e->GetClassification();
    if (HourlyClassification* hc = dynamic_cast<HourlyClassification*>(pc)) {
      hc->AddTimeCard(new TimeCard(itsDate, itsHours));
    } else
```

```
      throw("Tried to add timecard to non-hourly employee");
  } else
    throw("No such employee.");
}
```

Figures 19-9 and 19-10 show a similar design for the transaction that posts sales receipts to a commissioned employee. I've left the implementation of these classes as an exercise.

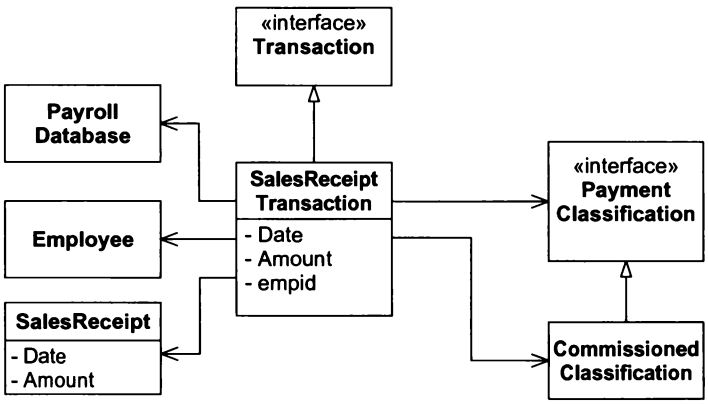


Figure 19-9 Static Model for `SalesReceiptTransaction`

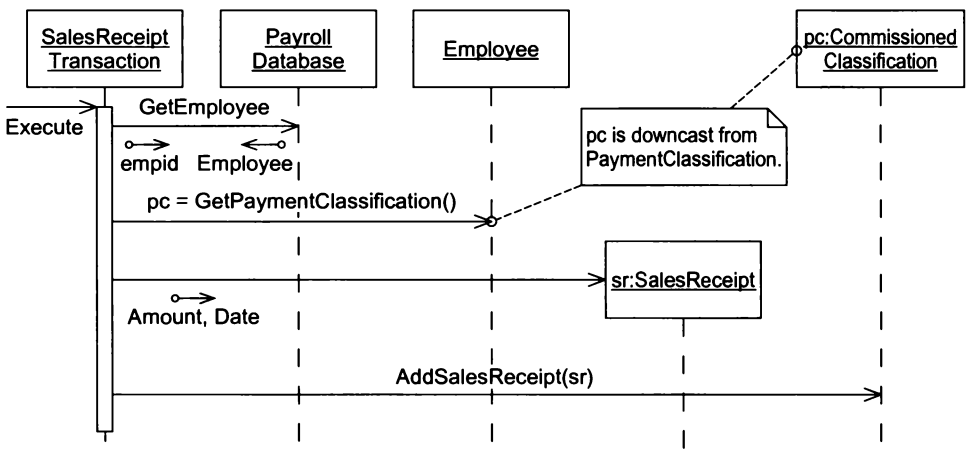


Figure 19-10 Dynamic Model for `SalesReceiptTransaction`

Figures 19-11 and 19-12 show the design for the transaction that posts service charges to union members.

These designs point out a mismatch between the transaction model and the core model that we have created. Our core `Employee` object can be affiliated with many different organizations, but the transaction model assumes that any affiliation must be a union affiliation. Thus, the transaction model provides no way to identify a particular kind of affiliation. Instead, it simply assumes that if we are posting a service charge, then the employee has a union affiliation.

The dynamic model addresses this dilemma by searching the set of `Affiliation` objects contained by the `Employee` object for a `UnionAffiliation` object. It then adds the `ServiceCharge` object to that `UnionAffiliation`.

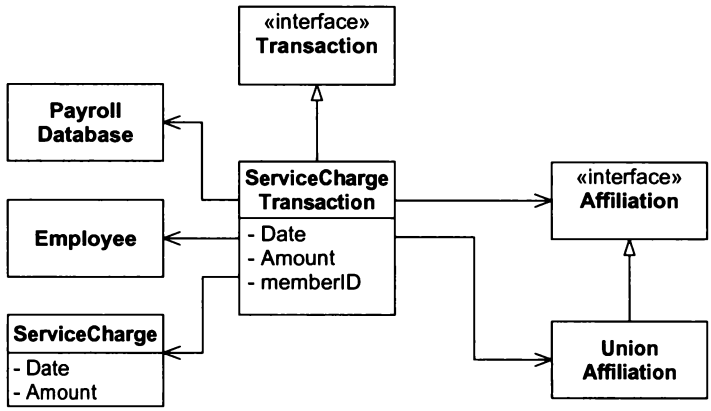


Figure 19-11 Static Model for `ServiceChargeTransaction`

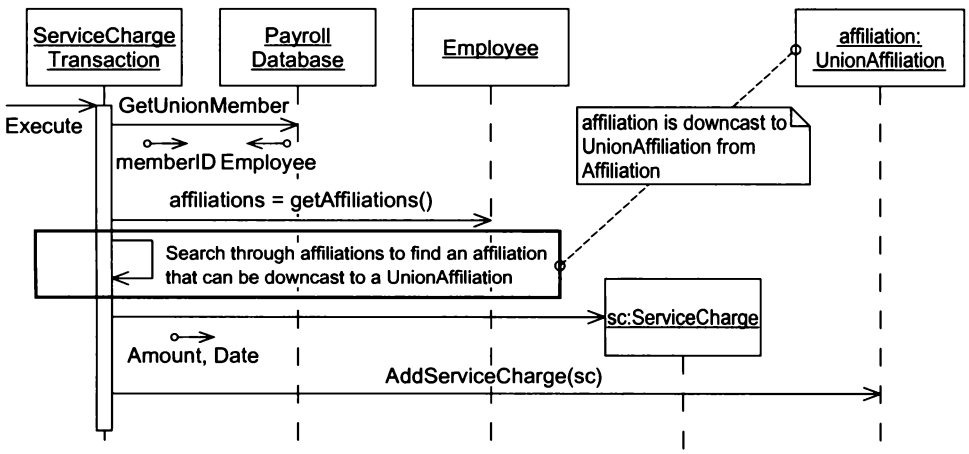


Figure 19-12 Dynamic Model for `ServiceChargeTransaction`

Listing 19-16 shows the test case for the `ServiceChargeTransaction`. It simply creates an hourly employee and adds a `UnionAffiliation` to it. It also makes sure that the appropriate member ID is registered with the `PayrollDatabase`. Then it creates a `ServiceChargeTransaction` and executes it. Finally it makes sure that the appropriate `ServiceCharge` was indeed added to `Employee`'s `UnionAffiliation`.

Listing 19-16

`PayrollTest::TestAddServiceCharge()`

```
void PayrollTest::TestAddServiceCharge()
{
  cerr << "TestAddServiceCharge" << endl;
  int empId = 2;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  Employee* e = GpayrollDatabase.GetEmployee(empId);
  assert(e);
  UnionAffiliation* af = new UnionAffiliation(12.5);
  e->SetAffiliation(af);
  int memberId = 86; // Maxwell Smart
  GpayrollDatabase.AddUnionMember(memberId, e);
  ServiceChargeTransaction sct(memberId, 20011101, 12.95);
```

```
  sct.Execute();
  ServiceCharge* sc = af->GetServiceCharge(20011101);
  assert(sc);
  assertEquals(12.95, sc->GetAmount(), .001);
}
```

Code v. UML. When I drew the UML in Figure 19-12, I thought that replacing `NoAffiliation` with a list of affiliations was a better design. I thought it was more flexible and less complex. After all, I could add new affiliations any time I wanted, and I didn't have to create the `NoAffiliation` class. However, when writing the test case in Listing 19-16, I realized that calling `SetAffiliation` on `Employee` was better than calling `AddAffiliation`. After all, the requirements do not ask that an employee have more than one `Affiliation`, so there is no need to employ `dynamic_cast` to select between potentially many kinds. Doing so would be more complex than necessary.

This is an example of why doing too much UML without verifying it in code can be dangerous. The code can tell you things about your design that the UML cannot. Here, I was putting structures into the UML that weren't needed. Maybe one day they'd come in handy, but they have to be maintained between now and then. The cost of that maintenance may not be worth the benefit.

In this case, even though the cost of maintaining the `dynamic_cast` is relatively slight, I'm not going to employ it. Its much simpler to implement without a list of `Affiliation` objects. So I'll keep the NULL OBJECT pattern in place with the `NoAffiliation` class.

Listings 19-17 and 19-18 show the implementation of the `ServiceChargeTransaction`. It is indeed much simpler without the loop looking for `UnionAffiliation` objects. It simply gets the `Employee` from the database, downcasts its `Affillation` to a `UnionAffilliation`, and adds the `ServiceCharge` to it.

Listing 19-17

ServiceChargeTransaction.h

```
#ifndef SERVICECHARGETRANSACTION_H
#define SERVICECHARGETRANSACTION_H

#include "Transaction.h"

class ServiceChargeTransaction : public Transaction
{
 public:
  virtual ~ServiceChargeTransaction();
  ServiceChargeTransaction(int memberId, long date, double charge);
  virtual void Execute();

 private:
  int itsMemberId;
  long itsDate;
  double itsCharge;
};
#endif
```

Listing 19-18

ServiceChargeTransaction.cpp

```
#include "ServiceChargeTransaction.h"
#include "Employee.h"
```

```
#include "ServiceCharge.h"
#include "PayrollDatabase.h"
#include "UnionAffiliation.h"

extern PayrollDatabase GpayrollDatabase;

ServiceChargeTransaction::~ServiceChargeTransaction()
{
}

ServiceChargeTransaction::
ServiceChargeTransaction(int memberId, long date, double charge)
:itsMemberId(memberId)
, itsDate(date)
, itsCharge(charge)
{
}

void ServiceChargeTransaction::Execute()
{
  Employee* e = GpayrollDatabase.GetUnionMember(itsMemberId);
  Affiliation* af = e->GetAffiliation();
  if (UnionAffiliation* uaf = dynamic_cast<UnionAffiliation*>(af)) {
    uaf->AddServiceCharge(itsDate, itsCharge);
  }
}
```

Changing Employees



Figures 19-13 and 19-14 show the static structure for the transactions that change the attributes of an employee. This structure is easily derived from Use Case 6. All the transactions take an `EmpID` argument, so we can create a top-level base class called `ChangeEmployeeTransaction`. Below this base class are the classes that change single attributes, such as `ChangeNameTransaction` and `ChangeAddressTransaction`. The transactions that change classifications have a commonality of purpose, in that they all modify the same field of the `Employee` object. Thus, they can be grouped together under the abstract base, `ChangeClassificationTransaction`. The same is true of the transactions that change the payment and the affiliations. This can be seen by the structure of `ChangeMethodTransaction` and `ChangeAffiliationTransaction`.

Figure 19-15 shows the dynamic model for all the change transactions. Again we see the TEMPLATE METHOD pattern in use. In every case, the `Employee` object corresponding to the `EmpID` must be retrieved from the `PayrollDatabase`. Thus, the `Execute` function of `ChangeEmployeeTransaction` implements this behavior and then sends the `Change` message to itself. This method will be declared as virtual and implemented in the derivatives, as shown in Figures 19-16 and 19-17.

Listing 19-19 shows the test case for the `ChangeNameTransaction`. This test case is very simple. It uses the `AddHourlyEmployee` transaction to create an hourly employee named Bill. It then creates and executes a `ChangeNameTransaction` that should change the employee's name to Bob. Finally, it fetches the `Employee` instance from the `PayrollDatabase` and verifies that the name has been changed.

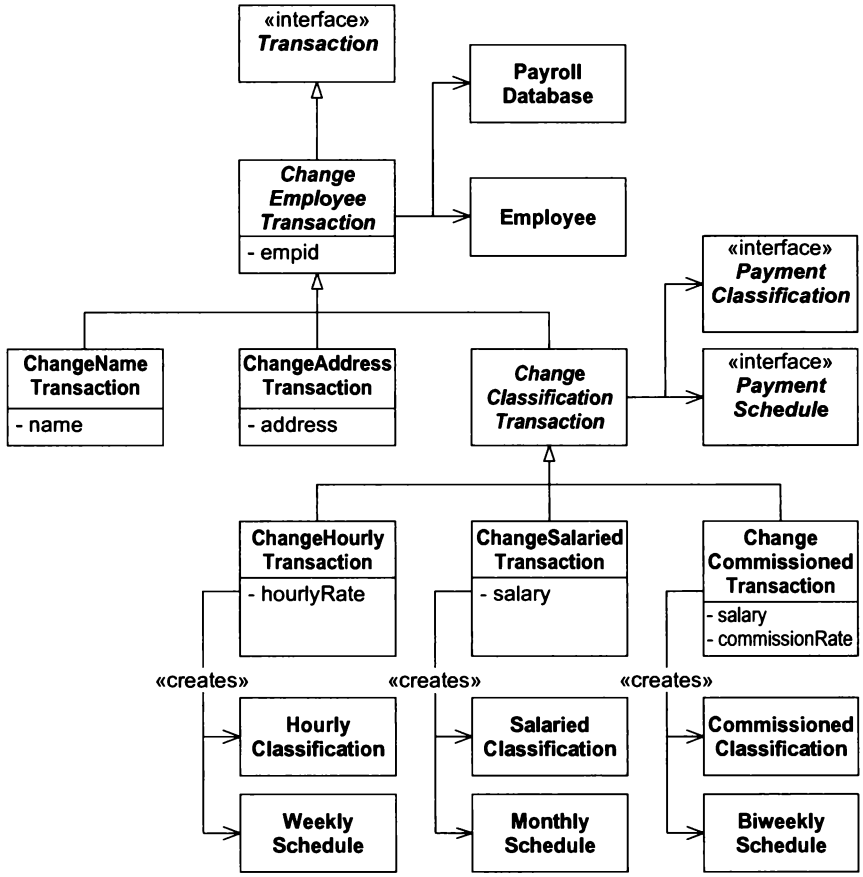


Figure 19-13 Static Model for `ChangeEmployeeTransaction`

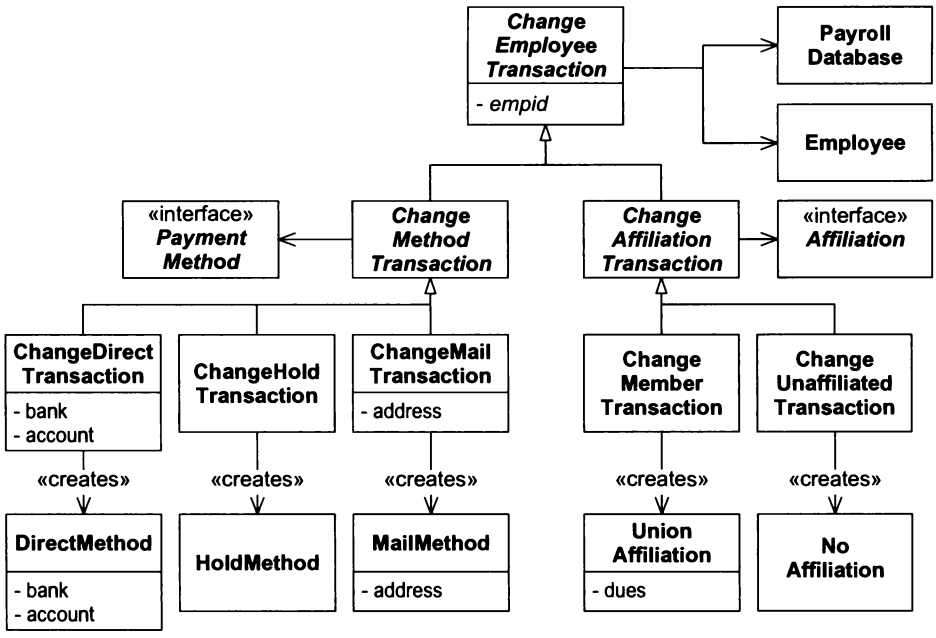


Figure 19-14 Static Model for `ChangeEmployeeTransaction` (cont.)

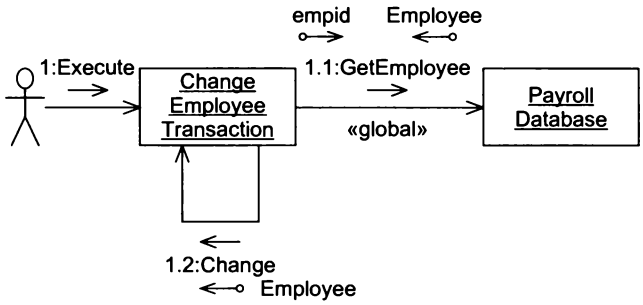


Figure 19-15 Dynamic Model for `ChangeEmployeeTransaction`

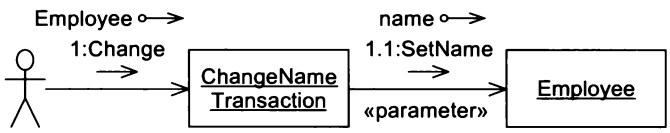


Figure 19-16 Dynamic Model for `ChangeNameTransaction`

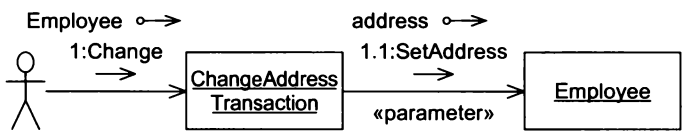


Figure 19-17 Dynamic Model for `ChangeAddressTransaction`

Listing 19-19

`PayrollTest::TestChangeNameTransaction()`

```
void PayrollTest::TestChangeNameTransaction()
{
  cerr << "TestChangeNameTransaction" << endl;
  int empId = 2;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  ChangeNameTransaction cnt(empId, "Bob");
  cnt.Execute();
  Employee* e = GpayrollDatabase.GetEmployee(empId);
  assert(e);
  assert("Bob" == e->GetName());
}
```

Listings 19-20 and 19-21 show the implementation of the abstract base class `ChangeEmployeeTransaction`. The structure of the TEMPLATE METHOD pattern is clearly in evidence. The `Execute()` method simply reads the appropriate `Employee` instance from the `PayrollDatabase` and, if successful, invokes the pure virtual `Change()` function.

Listing 19-20

`ChangeEmployeeTransaction.h`

```
#ifndef CHANGEEMPLOYEETRANSACTION_H
#define CHANGEEMPLOYEETRANSACTION_H

#include "Transaction.h"
#include "Employee.h"
```

```
class ChangeEmployeeTransaction : public Transaction
{
 public:
  ChangeEmployeeTransaction(int empid);
  virtual ~ChangeEmployeeTransaction();
  virtual void Execute();
  virtual void Change(Employee&) = 0;

 private:
  int itsEmpId;
};

#endif
```

Listing 19-21

ChangeEmployeeTransaction.cpp

```
#include "ChangeEmployeeTransaction.h"
#include "Employee.h"
#include "PayrollDatabase.h"

extern PayrollDatabase GpayrollDatabase;

ChangeEmployeeTransaction::~ChangeEmployeeTransaction()
{
}

ChangeEmployeeTransaction::ChangeEmployeeTransaction(int empid)
: itsEmpId(empid)
{
}

void ChangeEmployeeTransaction::Execute()
{
  Employee* e = GpayrollDatabase.GetEmployee(itsEmpId);
  if (e != 0)
    Change(*e);
}
```

Listings 19-22 and 19-23 show the implementation of the `ChangeNameTransaction`. The second half of the TEMPLATE METHOD can easily be seen. The `Change()` method is implemented to change the name of the `Employee` argument. The structure of the `ChangeAddressTransaction` is very similar and is left as an exercise.

Listing 19-22

ChangeEmployeeTransaction.h

```
#ifndef CHANGENAMETRANSACTION_H
#define CHANGENAMETRANSACTION_H

#include "ChangeEmployeeTransaction.h"
#include <string>
```

```
class ChangeNameTransaction : public ChangeEmployeeTransaction
{
 public:
  virtual ~ChangeNameTransaction();
  ChangeNameTransaction(int empid, string name);
  virtual void Change(Employee&);

 private:
  string itsName;
};

#endif
```

Listing 19-23

ChangeNameTransaction.cpp

```
#include "ChangeNameTransaction.h"

ChangeNameTransaction::~ChangeNameTransaction()
{
}

ChangeNameTransaction::ChangeNameTransaction(int empid,
                                             string name)
: ChangeEmployeeTransaction(empid)
, itsName(name)
{
}

void ChangeNameTransaction::Change(Employee& e)
{
  e.SetName(itsName);
}
```

Changing Classification

Figure 19-18 shows how the dynamic behavior of `ChangeClassificationTransaction` is envisioned. The TEMPLATE METHOD pattern is used yet again. The transactions must create a new `PaymentClassification` object and then hand it to the `Employee` object. This is accomplished by sending the `GetClassification` mes-

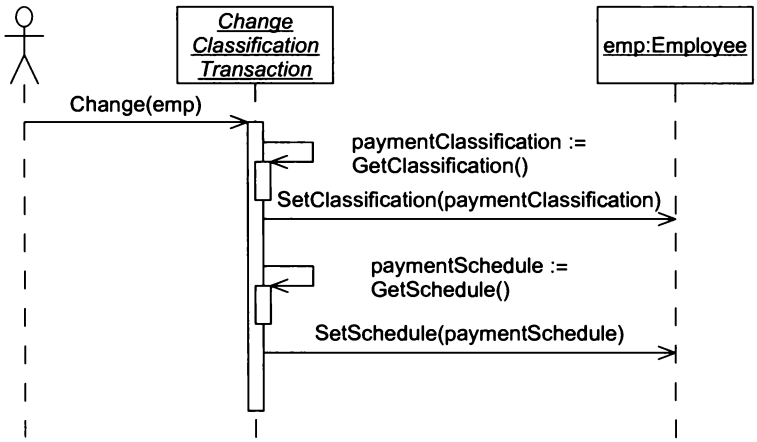


Figure 19-18 Dynamic Model of the `ChangeClassificationTransaction`

sage to itself. This abstract method is implemented in each of the classes derived from `ChangeClassificationTransaction`, as shown in Figures 19-19 through 19-21.

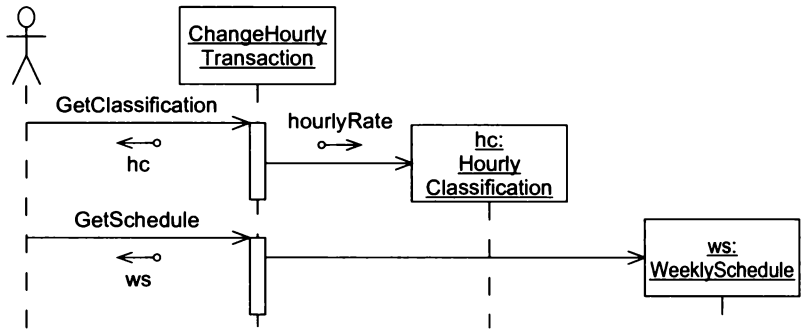


Figure 19-19 Dynamic Model of `ChangeHourlyTransaction`

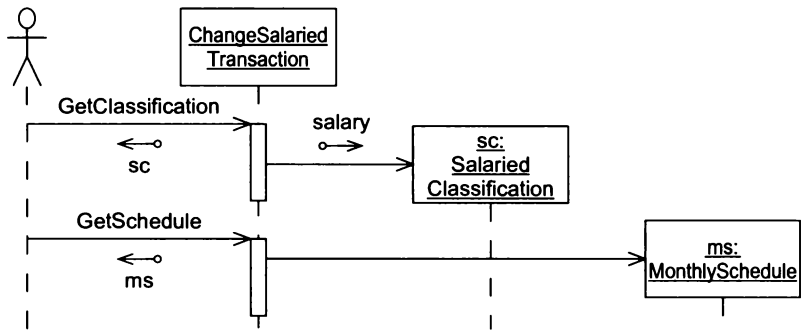


Figure 19-20 Dynamic Model of `ChangeSalariedTransaction`

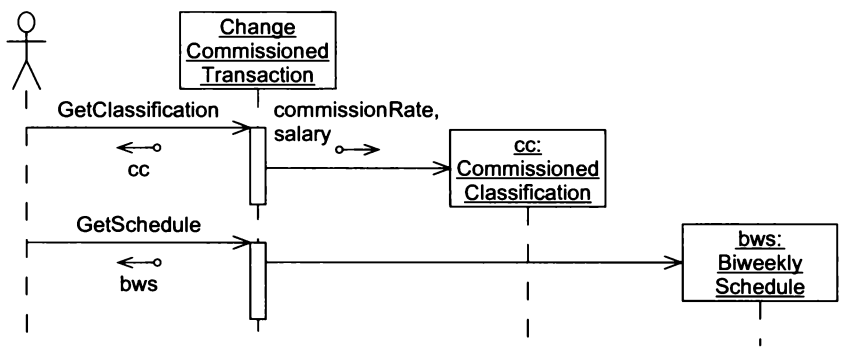


Figure 19-21 Dynamic Model of `ChangeCommissionedTransaction`

Listing 19-24 shows the test case for `ChangeHourlyTransaction`. The test case uses an `AddCommissionedEmployee` transaction to create a commissioned employee. It then creates a `ChangeHourlyTransaction` and executes it. It fetches the changed employee and verifies that its `PaymentClassification` is an `HourlyClassification` with the appropriate hourly rate and that its `PaymentSchedule` is a `WeeklySchedule`.

Listing 19-24

PayrollTest::TestChangeHourlyTransaction()

```
void PayrollTest::TestChangeHourlyTransaction()
{
  cerr << "TestChangeHourlyTransaction" << endl;
  int empId = 3;
  AddCommissionedEmployee t(empId, "Lance", "Home", 2500, 3.2);
  t.Execute();
  ChangeHourlyTransaction cht(empId, 27.52);
  cht.Execute();
  Employee* e = GpayrollDatabase.GetEmployee(empId);
  assert(e);
  PaymentClassification* pc = e->GetClassification();
  assert(pc);
  HourlyClassification* hc =
    dynamic_cast<HourlyClassification*>(pc);
  assert(hc);
  assertEquals(27.52, hc->GetRate(), .001);
  PaymentSchedule* ps = e->GetSchedule();
  WeeklySchedule* ws = dynamic_cast<WeeklySchedule*>(ps);
  assert(ws);
}
```

Listings 19-25 and 19-26 show the implementation of the abstract base class `ChangeClassificationTransaction`. Once again, the TEMPLATE METHOD pattern is easy to pick out. The `Change()` method invokes the two pure virtual functions, `GetClassification()` and `GetSchedule()`. It uses the return values from these functions to set the classification and schedule of the `Employee`.

Listing 19-25

ChangeClassificationTransaction.h

```
#ifndef CHANGECLASSIFICATIONTRANSACTION_H
#define CHANGECLASSIFICATIONTRANSACTION_H

#include "ChangeEmployeeTransaction.h"

class PaymentClassification;
class PaymentSchedule;

class ChangeClassificationTransaction : public ChangeEmployeeTransaction
{
 public:
  virtual ~ChangeClassificationTransaction();
  ChangeClassificationTransaction(int empid);
  virtual void Change(Employee&);
  virtual PaymentClassification* GetClassification() const = 0;
  virtual PaymentSchedule* GetSchedule() const = 0;
};
#endif
```

Listing 19-26

ChangeClassificationTransaction.cpp

```
#include "ChangeClassificationTransaction.h"

ChangeClassificationTransaction::~ChangeClassificationTransaction()
{
}

ChangeClassificationTransaction::ChangeClassificationTransaction(int empid)
: ChangeEmployeeTransaction(empid)
{
}

void ChangeClassificationTransaction::Change(Employee& e)
{
  e.SetClassification(GetClassification());
  e.SetSchedule(GetSchedule());
}
```

Listings 19-27 and 19-28 show the implementation of the `ChangeHourlyTransaction` class. This class completes the TEMPLATE METHOD pattern by implementing the `GetClassification()` and `GetSchedule()` methods that it inherited from `ChangeClassificationTransaction`. It implements `GetClassification()` to return a newly created `HourlyClassification`. It implements `GetSchedule()` to return a newly created `WeeklySchedule`.

Listing 19-27

ChangeHourlyTransaction.h

```
#ifndef CHANGEHOURLYTRANSACTION_H
#define CHANGEHOURLYTRANSACTION_H

#include "ChangeClassificationTransaction.h"

class ChangeHourlyTransaction : public ChangeClassificationTransaction
{
 public:
  virtual ~ChangeHourlyTransaction();
  ChangeHourlyTransaction(int empid, double hourlyRate);
  virtual PaymentSchedule* GetSchedule() const;
  virtual PaymentClassification* GetClassification() const;

 private:
  double itsHourlyRate;
};

#endif
```

Listing 19-28

ChangeHourlyTransaction.cpp

```
#include "ChangeHourlyTransaction.h"
#include "WeeklySchedule.h"
#include "HourlyClassification.h"
```

```
ChangeHourlyTransaction::~ChangeHourlyTransaction()
{
}

ChangeHourlyTransaction::ChangeHourlyTransaction(int empid, double hourlyRate)
: ChangeClassificationTransaction(empid)
, itsHourlyRate(hourlyRate)
{
}

PaymentSchedule* ChangeHourlyTransaction::GetSchedule() const
{
  return new WeeklySchedule();
}

PaymentClassification* ChangeHourlyTransaction::GetClassification() const
{
  return new HourlyClassification(itsHourlyRate);
}
```

As always, the `ChangeSalariedTransaction` and `ChangeCommissionedTransaction` are left to the reader as an exercise.

A similar mechanism is employed for the implementation of `ChangeMethodTransaction`. The abstract `GetMethod` method is used to select the proper derivative of `PaymentMethod`, which is then handed to the `Employee` object. (See Figures 19-22 through 19-25.)

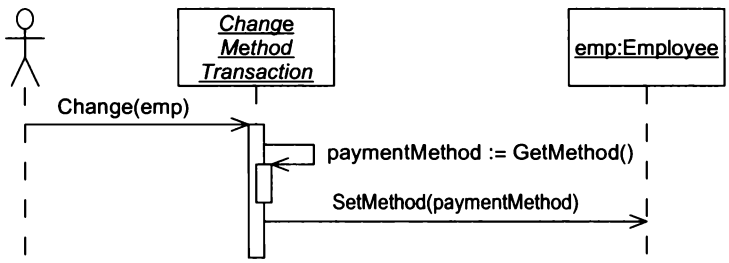


Figure 19-22 Dynamic Model of `ChangeMethodTransaction`

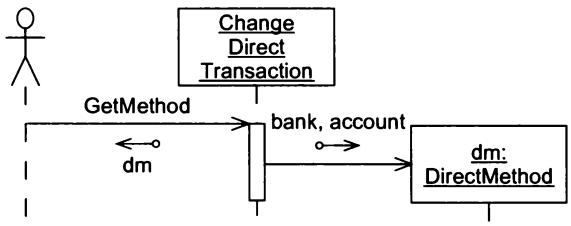


Figure 19-23 Dynamic Model of `ChangeDirectTransaction`

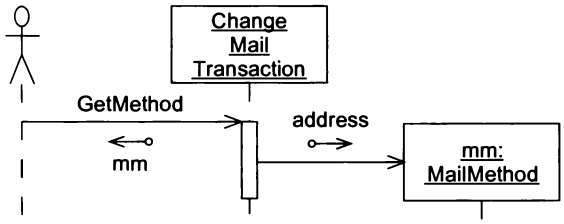


Figure 19-24 Dynamic Model of `ChangeMailTransaction`

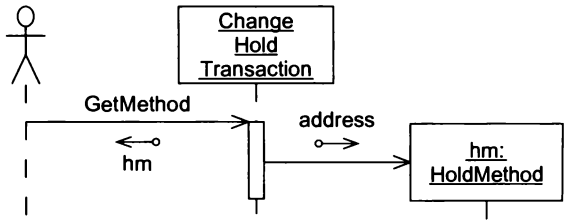


Figure 19-25 Dynamic Model of `ChangeHoldTransaction`

The implementation of these classes turned out to be straightforward and unsurprising. They too are left as an exercise.

Figure 19-26 shows the implementation of the `ChangeAffiliationTransaction`. Once again, we use the TEMPLATE METHOD pattern to select the `Affiliation` derivative that should be handed to the `Employee` object. (See Figures 19-27 through 19-29.)

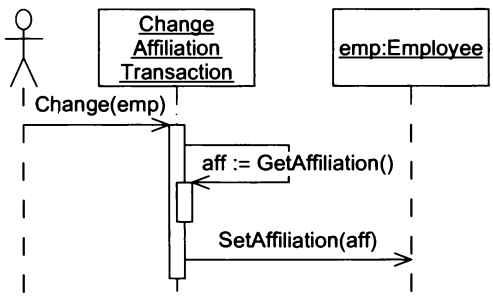


Figure 19-26 Dynamic Model of `ChangeAffiliationTransaction`

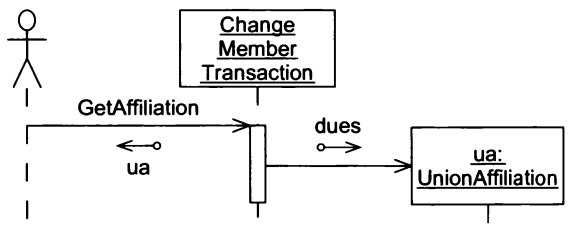


Figure 19-27 Dynamic Model of `ChangeMemberTransaction`

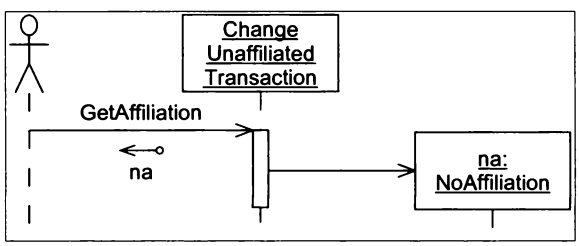


Figure 19-28 Dynamic Model of `ChangeUnaffiliatedTransaction`

What Was I Smoking?

I got quite a surprise when I went to implement this design. Look closely at the dynamic diagrams for the affiliation transactions. Can you spot the problem?

I began the implementation, as always, by writing the test case for `ChangeMemberTransaction`. You can see this test case in Listing 19-29. The test case starts out straightforward enough. It creates an hourly employee named Bill and then creates and executes a `ChangeMemberTransaction` to put Bill in the union. Then it checks to see that Bill has a `UnionAffiliation` bound to him and that the `UnionAffiliation` has the right dues rate.

Listing 19-29

PayrollTest::TestChangeMemberTransaction()

```
void PayrollTest::TestChangeMemberTransaction()
{
  cerr << "TestChangeMemberTransaction" << endl;
  int empId = 2;
  int memberId = 7734;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  ChangeMemberTransaction cmt(empId, memberId, 99.42);
  cmt.Execute();
  Employee* e = GpayrollDatabase.GetEmployee(empId);
  assert(e);
  Affiliation* af = e->GetAffiliation();
  assert(af);
  UnionAffiliation* uf = dynamic_cast<UnionAffiliation*>(af);
  assert(uf);
  assertEquals(99.42, uf->GetDues(), .001);
  Employee* member = GpayrollDatabase.GetUnionMember(memberId);
  assert(member);
  assert(e == member);
}
```

The surprise is hidden in the last few lines of the test case. Those lines make sure that the `PayrollDatabase` has recorded Bill's membership in the union. Nothing in the existing UML diagrams makes sure this happens. The UML is only concerned with the appropriate `Affiliation` derivative being bound to the `Employee`. I didn't notice the deficit at all. Did you?

I merrily coded the transactions as per the diagrams and then watched the unit test fail. Once the failure occurred, it was obvious what I had neglected. What was not obvious was the solution to the problem. How do I get the membership to be recorded by `ChangeMemberTransaction`, but erased by `ChangeUnaffiliatedTransaction`?

The answer was to add another pure virtual function to `ChangeAffiliationTransaction` named `RecordMembership(Employee*)`. This function is implemented in `ChangeMemberTransaction` to bind the `memberId` to the `Employee` instance. In the `ChangeUnaffiliatedTransaction` it is implemented to erase the membership record.

Listings 19-30 and 19-31 show the resulting implementation of the abstract base class `ChangeAffiliationTransaction`. Again, the use of the TEMPLATE METHOD pattern is obvious.

Listing 19-30

ChangeAffiliationTransaction.h

```
#ifndef CHANGEAFFILIATIONTRANSACTION_H
#define CHANGEAFFILIATIONTRANSACTION_H

#include "ChangeEmployeeTransaction.h"

class ChangeAffiliationTransaction: public ChangeEmployeeTransaction
{
 public:
  virtual ~ChangeAffiliationTransaction();
```

```
  ChangeAffiliationTransaction(int empid);
  virtual Affiliation* GetAffiliation() const = 0;
  virtual void RecordMembership(Employee*) = 0;
  virtual void Change(Employee&);
};

#endif
```

Listing 19-31

ChangeAffiliationTransaction.cpp

```
#include "ChangeAffiliationTransaction.h"

ChangeAffiliationTransaction::~ChangeAffiliationTransaction()
{
}

ChangeAffiliationTransaction::ChangeAffiliationTransaction(int empid)
: ChangeEmployeeTransaction(empid)
{
}

void ChangeAffiliationTransaction::Change(Employee& e)
{
  RecordMembership(&e);
  e.SetAffiliation(GetAffiliation());
}
```

Listing 19-32 and Listing 19-33 show the implementation of `ChangeMemberTransaction`. This is not particularly complicated or interesting. On the other hand, the implementation of `ChangeUnaffiliatedTransaction` in Listings 19-34 and 19-35 is a bit more substantial. The `RecordMembership` function has to decide whether or not the current employee is a union member. If so, it then gets the `memberId` from the `UnionAffiliation` and erases the membership record.

Listing 19-32

ChangeMemberTransaction.h

```
#ifndef CHANGEMEMBERTRANSACTION_H
#define CHANGEMEMBERTRANSACTION_H

#include "ChangeAffiliationTransaction.h"

class ChangeMemberTransaction : public ChangeAffiliationTransaction
{
 public:
  virtual ~ChangeMemberTransaction();
  ChangeMemberTransaction(int empid, int memberid, double dues);
  virtual Affiliation* GetAffiliation() const;
  virtual void RecordMembership(Employee*);
 private:
  int itsMemberId;
  double itsDues;
};
#endif
```

Listing 19-33

ChangeMemberTransaction.cpp

```
#include "ChangeMemberTransaction.h"
#include "UnionAffiliation.h"
#include "PayrollDatabase.h"

extern PayrollDatabase GpayrollDatabase;

ChangeMemberTransaction::~ChangeMemberTransaction()
{
}

ChangeMemberTransaction::
ChangeMemberTransaction(int empid, int memberid, double dues)
: ChangeAffiliationTransaction(empid)
, itsMemberId(memberid)
, itsDues(dues)
{
}

Affiliation* ChangeMemberTransaction::GetAffiliation() const
{
  return new UnionAffiliation(itsMemberId, itsDues);
}

void ChangeMemberTransaction::RecordMembership(Employee* e)
{
  GpayrollDatabase.AddUnionMember(itsMemberId, e);
}
```

Listing 19-34

ChangeUnaffiliatedTransaction.h

```
#ifndef CHANGEUNAFFILIATEDTRANSACTION_H
#define CHANGEUNAFFILIATEDTRANSACTION_H

#include "ChangeAffiliationTransaction.h"

class ChangeUnaffiliatedTransaction : public ChangeAffiliationTransaction
{
 public:
  virtual ~ChangeUnaffiliatedTransaction();
  ChangeUnaffiliatedTransaction(int empId);
  virtual Affiliation* GetAffiliation() const;
  virtual void RecordMembership(Employee*);
};
#endif
```

Listing 19-35

ChangeUnaffiliatedTransaction.cpp

```
#include "ChangeUnaffiliatedTransaction.h"
#include "NoAffiliation.h"
#include "UnionAffiliation.h"
#include "PayrollDatabase.h"

extern PayrollDatabase GpayrollDatabase;
```

```
ChangeUnaffiliatedTransaction::~ChangeUnaffiliatedTransaction()
{
}

ChangeUnaffiliatedTransaction::ChangeUnaffiliatedTransaction(int empId)
: ChangeAffiliationTransaction(empId)
{
}

Affiliation* ChangeUnaffiliatedTransaction::GetAffiliation() const
{
  return new NoAffiliation();
}

void ChangeUnaffiliatedTransaction::RecordMembership(Employee* e)
{
  Affiliation* af = e->GetAffiliation();
  if (UnionAffiliation* uf = dynamic_cast<UnionAffiliation*>(af))
  {
    int memberId = uf->GetMemberId();
    GpayrollDatabase.RemoveUnionMember(memberId);
  }
}
```

I can't say that I'm very pleased with this design. It bothers me that the `ChangeUnaffiliatedTransaction` must know about `UnionAffiliation`. I could solve this by putting `RecordMembership` and `EraseMembership` abstract methods in the `Affiliation` class. However, this would force `UnionAffiliation` and `NoAffiliation` to know about the `PayrollDatabase`. And I'm not very happy about that either.[2]

Still, the implementation as it stands is pretty simple and only slightly violates the OCP. The nice thing is that very few modules in the system know about `ChangeUnaffiliatedTransaction`, so its extra dependencies aren't doing very much harm.

Paying Employees



Finally, it is time to consider the transaction that is at the root of this application: the transaction that instructs the system to pay the appropriate employees. Figure 19-29 shows the static structure of the `PaydayTransaction` class. Figures 19-30 through 19-33 describe the dynamic behavior.

These few dynamic models express a great deal of polymorphic behavior. The algorithm employed by the `CalculatePay` message depends on the kind of `PaymentClassification` that the `employee` object contains. The algorithm used to determine if a date is a payday depends on the kind of `PaymentSchedule` that the `Employee` contains. The algorithm used to send the payment to the `Employee` depends on the type of the `PaymentMethod` object. This high degree of abstraction allows the algorithms to be closed against the addition of new kinds of payment classifications, schedules, affiliations, or payment methods.

2. I could use the VISITOR pattern (page 387) to solve this problem, but that would probably be way overengineered.

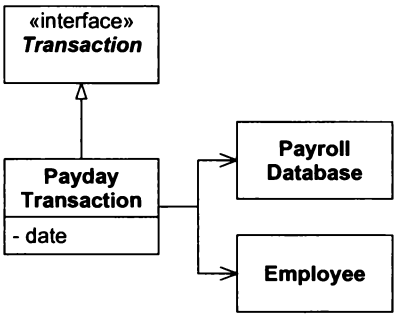


Figure 19-29 Static Model of `PaydayTransaction`

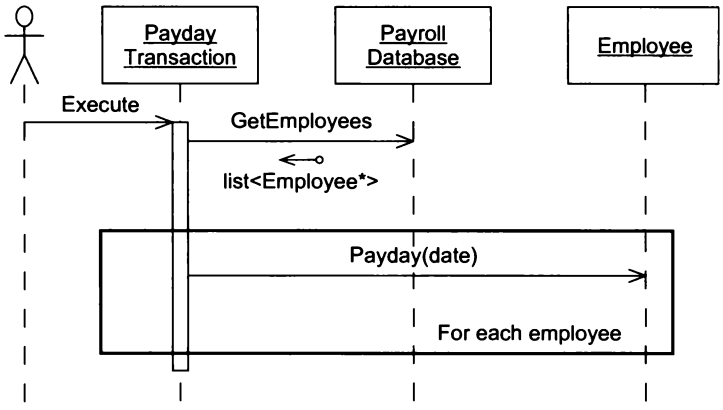


Figure 19-30 Dynamic Model for `PaydayTransaction`

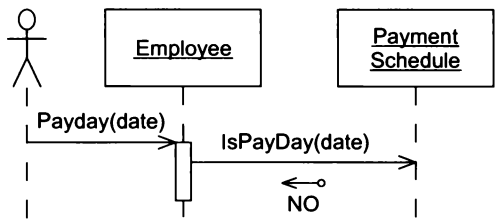


Figure 19-31 Dynamic Model Scenario: "Payday is not today."

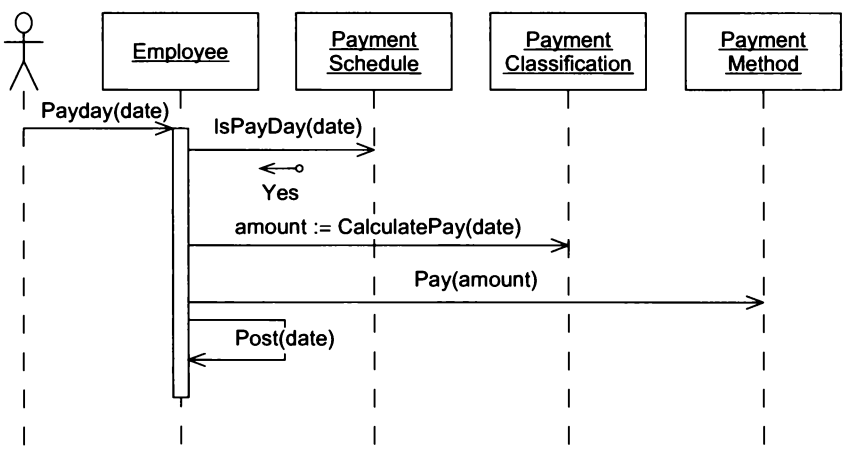


Figure 19-32 Dynamic Model Scenario: "Payday is today."

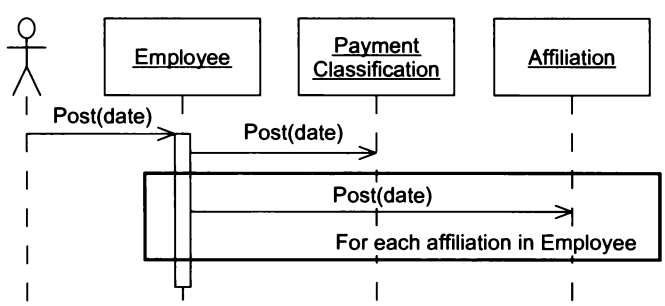


Figure 19-33 Dynamic Model Scenario: Posting Payment

The algorithms depicted in Figure 19-32 and Figure 19-33 introduce the concept of *posting*. After the correct pay amount has been calculated and sent to the `Employee`, the payment is posted; that is, the records involved in the payment are updated. Thus, we can define the `CalculatePay` method as calculating the pay from the last posting until the specified date.

Do We Want Developers Making Business Decisions?

Where did this notion of posting come from? It certainly wasn't mentioned in the user stories or use cases. As it happens, I cooked it up as a way to solve a problem that I perceived. I was concerned that the `Payday` method might be called multiple times with the same date, or with a date in the same pay period, so I wanted to make sure that the employee was not paid more than once. I did this on my own initiative, without asking my customer. It just seemed the right thing to do.

In effect, I have made a business decision. I have decided that multiple runs of the payroll program will produce different results. I should have asked my customer or project manager about this, since they might have very different ideas.

In checking with the customer, I find that the idea of posting goes against his intent.[3] The customer wants to be able to run the payroll system and then review the paychecks. If any of them are wrong, the customer wants to correct the payroll information and run the payroll program again. They tell me that I should never consider time cards or sales receipts for dates outside the current pay period.

So, we have to ditch the posting scheme. It seemed like a good idea at the time, but it was not what the customer wanted.

Paying Salaried Employees

There are two test cases in Listing 19-36. They test whether a salaried employee is being paid appropriately. The first test case makes sure the employee is paid on the last day of the month. The second test case makes sure the employee is not paid if it is not the last day of the month.

Listing 19-36

`PayrollTest::TestPaySingleSalariedEmployee & co.`

```
void PayrollTest::TestPaySingleSalariedEmployee()
{
  cerr << "TestPaySingleSalariedEmployee" << endl;
  int empId = 1;
  AddSalariedEmployee t(empId, "Bob", "Home", 1000.00);
  t.Execute();
  Date payDate(11,30,2001);
  PaydayTransaction pt(payDate);
  pt.Execute();
  Paycheck* pc = pt.GetPaycheck(empId);
  assert(pc);
  assert(pc->GetPayDate() == payDate);
  assertEquals(1000.00, pc->GetGrossPay(), .001);
  assert("Hold" == pc->GetField("Disposition"));
  assertEquals(0.0, pc->GetDeductions(), .001);
  assertEquals(1000.00, pc->GetNetPay(), .001);
}
```

3. OK, I am the customer.

```
void PayrollTest::TestPaySingleSalariedEmployeeOnWrongDate()
{
  cerr << "TestPaySingleSalariedEmployeeWrongDate" << endl;
  int empId = 1;
  AddSalariedEmployee t(empId, "Bob", "Home", 1000.00);
  t.Execute();
  Date payDate(11,29,2001);
  PaydayTransaction pt(payDate);
  pt.Execute();
  Paycheck* pc = pt.GetPaycheck(empId);
  assert(pc == 0);
}
```

Remember back in Listing 19-13, when I was implementing the `TimeCard` class, I used a long integer to represent the date? Well, now I have a need for a real `Date` class. These two test cases will not pass unless I can tell whether the pay date is the last day of the month.

It turns out that I wrote a `Date` class about 10 years ago for a C++ class I was teaching. So I dug through my archives and found it on an old sparcstation that I had laying around.[4] I moved it to my development environment and managed to get it to compile in minutes. I found this surprising, since I had written it to work in Linux, but was now using it in Windows 2000. There were a couple of small bugs to fix, and I had to replace my homegrown string class with the STL string class, but in the end the effort was minimal.

Listing 19-37 shows the `Execute()` function of `PaydayTransaction`. It iterates through all the `Employee` objects in the database. It asks each employee if the date on this transaction is its pay date. If so, it creates a new paycheck for the employee and tells the employee to fill in its fields.

Listing 19-37

`PaydayTransaction::Execute()`

```
void PaydayTransaction::Execute()
{
  list<int> empIds;
  GpayrollDatabase.GetAllEmployeeIds(empIds);

  list<int>::iterator i = empIds.begin();
  for (; i != empIds.end(); i++) {
    int empId = *i;
    if (Employee* e = GpayrollDatabase.GetEmployee(empId)) {
      if (e->IsPayDate(itsPayDate)) {
        Paycheck* pc = new Paycheck(itsPayDate);
        itsPaychecks[empId] = pc;
        e->Payday(*pc);
      }
    }
  }
}
```

Listing 19-38 shows a fragment of `MonthlySchedule.cpp`. Notice that it implements `IsPayDate` to return `true` only if the argument date is the last day of the month. This algorithm points out why I needed the `Date` class. Doing this kind of simple date calculation is very difficult without a good `Date` class.

4. The original oma.com. This was a sparcstation that I purchased for $6,000 from a company that had bought it for a project and then cancelled the project. Back in 1994 that was a real good deal. The fact that the machine is still quietly running on the Object Mentor network is a testimony to how well built it was.

Listing 19-38

MonthlySchedule.cpp (fragment)

```
namespace
{
  bool IsLastDayOfMonth(const Date& date)
  {
    int m1 = date.GetMonth();
    int m2 = (date+1).GetMonth();
    return (m1 != m2);
  }
}

bool MonthlySchedule::IsPayDate(const Date& payDate) const
{
  return IsLastDayOfMonth(payDate);
}
```

Listing 19-39 shows the implementation of `Employee::PayDay()`. This function is the generic algorithm for calculating and dispatching payment for all employees. Notice the rampant use of the STRATEGY pattern. All detailed calculations are deferred to the contained strategy classes: `itsClassification`, `itsAffiliation`, and `itsPaymentMethod`.

Listing 19-39

Employee::PayDay()

```
void Employee::Payday(Paycheck& pc)
{
  double grossPay = itsClassification->CalculatePay(pc);
  double deductions = itsAffiliation->CalculateDeductions(pc);
  double netPay = grossPay - deductions;
  pc.SetGrossPay(grossPay);
  pc.SetDeductions(deductions);
  pc.SetNetPay(netPay);
  itsPaymentMethod->Pay(pc);
}
```

Paying Hourly Employees

Getting the hourly employees paid is a good example of the incrementalism of test-first design. I started with very trivial test cases and worked my way up to ever more complex ones. I'll show the test cases below, and then I'll show you the production code that resulted from them.

Listing 19-40 shows the simplest case. We add an hourly employee to the database and then pay him. Since there aren't any time cards, we expect the paycheck to have a zero value. The utility function `ValidateHourlyPaycheck` represents a refactoring that happened later. At first, that code was simply buried inside the test function. This test case worked without making any changes to the rest of the code.

Listing 19-40

TestPaySingleHourlyEmployeeNoTimeCards

```
void PayrollTest::TestPaySingleHourlyEmployeeNoTimeCards()
{
  cerr << "TestPaySingleHourlyEmployeeNoTimeCards" << endl;
  int empId = 2;
```

```
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  Date payDate(11,9,2001); // Friday
  PaydayTransaction pt(payDate);
  pt.Execute();
  ValidateHourlyPaycheck(pt, empId, payDate, 0.0);
}

void PayrollTest::ValidateHourlyPaycheck(PaydayTransaction& pt,
                                         int empid,
                                         const Date& payDate,
                                         double pay)
{
  Paycheck* pc = pt.GetPaycheck(empid);
  assert(pc);
  assert(pc->GetPayDate() == payDate);
  assertEquals(pay, pc->GetGrossPay(), .001);
  assert("Hold" == pc->GetField("Disposition"));
  assertEquals(0.0, pc->GetDeductions(), .001);
  assertEquals(pay, pc->GetNetPay(), .001);
}
```

Listing 19-41 shows two test cases. The first tests whether we can pay an employee after adding a single time card. The second tests whether we can pay overtime for a card that has more than 8 hours on it. Of course, I didn't write these two test cases at the same time. Instead, I wrote the first one and got it working, and then I wrote the second one.

Listing 19-41

Test...OneTimeCard

```
void PayrollTest::TestPaySingleHourlyEmployeeOneTimeCard()
{
  cerr << "TestPaySingleHourlyEmployeeOneTimeCard" << endl;
  int empId = 2;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  Date payDate(11,9,2001); // Friday

  TimeCardTransaction tc(payDate, 2.0, empId);
  tc.Execute();
  PaydayTransaction pt(payDate);
  pt.Execute();
  ValidateHourlyPaycheck(pt, empId, payDate, 30.5);
}

void
PayrollTest::TestPaySingleHourlyEmployeeOvertimeOneTimeCard()
{
  cerr << "TestPaySingleHourlyEmployeeOvertimeOneTimeCard" << endl;
  int empId = 2;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  Date payDate(11,9,2001); // Friday

  TimeCardTransaction tc(payDate, 9.0, empId);
  tc.Execute();
```

```
  PaydayTransaction pt(payDate);
  pt.Execute();
  ValidateHourlyPaycheck(pt, empId, payDate, (8 + 1.5) * 15.25);
}
```

Getting the first test case working was a matter of changing `HourlyClassification::CalculatePay` to loop through the time cards for the employee, add up the hours, and multiply by the pay rate. Getting the second test working forced me to refactor the function to calculate straight and overtime hours.

The test case in Listing 19-42 makes sure that we don't pay hourly employees unless the `PaydayTransaction` is constructed with a Friday.

Listing 19-42

`TestPaySingleHourlyEmployeeOnWrongDate`

```
void PayrollTest::TestPaySingleHourlyEmployeeOnWrongDate()
{
  cerr << "TestPaySingleHourlyEmployeeOnWrongDate" << endl;
  int empId = 2;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  Date payDate(11,8,2001); // Thursday

  TimeCardTransaction tc(payDate, 9.0, empId);
  tc.Execute();
  PaydayTransaction pt(payDate);
  pt.Execute();

  Paycheck* pc = pt.GetPaycheck(empId);
  assert(pc == 0);
}
```

Listing 19-43 is a test case that makes sure we can calculate the pay for an employee who has more than one time card.

Listing 19-43

`TestPaySingleHourlyEmployeeTwoTimeCards`

```
void PayrollTest::TestPaySingleHourlyEmployeeTwoTimeCards()
{
  cerr << "TestPaySingleHourlyEmployeeTwoTimeCards" << endl;
  int empId = 2;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  Date payDate(11,9,2001); // Friday

  TimeCardTransaction tc(payDate, 2.0, empId);
  tc.Execute();
  TimeCardTransaction tc2(Date(11,8,2001), 5.0, empId);
  tc2.Execute();
  PaydayTransaction pt(payDate);
  pt.Execute();
  ValidateHourlyPaycheck(pt, empId, payDate, 7*15.25);
}
```

Finally, the test case in Listing 19-44 proves that we will only pay an employee for time cards in the current pay period. Time cards from other pay periods are ignored.

Listing 19-44

TestPaySingleHourlyEmployeeWithTimeCardsSpanningTwoPayPeriods

```
void PayrollTest::
TestPaySingleHourlyEmployeeWithTimeCardsSpanningTwoPayPeriods()
{
  cerr << "TestPaySingleHourlyEmployeeWithTimeCards"
          "SpanningTwoPayPeriods" << endl;
  int empId = 2;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  Date payDate(11,9,2001); // Friday
  Date dateInPreviousPayPeriod(11,2,2001);

  TimeCardTransaction tc(payDate, 2.0, empId);
  tc.Execute();
  TimeCardTransaction tc2(dateInPreviousPayPeriod, 5.0, empId);
  tc2.Execute();
  PaydayTransaction pt(payDate);
  pt.Execute();
  ValidateHourlyPaycheck(pt, empId, payDate, 2*15.25);
}
```

The code that makes all this work was grown incrementally, one test case at a time. The structure you see in the code that follows evolved from test case to test case. Listing 19-45 shows the appropriate fragments of `HourlyClassification.cpp`. We simply loop through the time cards. For each time card, we check to see if it is in the pay period. If so, then we calculate the pay it represents.

Listing 19-45

HourlyClassification.cpp (Fragment)

```
double HourlyClassification::CalculatePay(Paycheck& pc) const
{
  double totalPay = 0;
  Date payPeriod = pc.GetPayDate();
  map<Date, TimeCard*>::const_iterator i;
  for (i=itsTimeCards.begin(); i != itsTimeCards.end(); i++) {
    TimeCard * tc = (*i).second;
    if (IsInPayPeriod(tc, payPeriod))
      totalPay += CalculatePayForTimeCard(tc);
  }
  return totalPay;
}

bool HourlyClassification::IsInPayPeriod(TimeCard* tc, const Date& payPeriod) const
{
  Date payPeriodEndDate = payPeriod;
  Date payPeriodStartDate = payPeriod - 5;
  Date timeCardDate = tc->GetDate();
  return (timeCardDate >= payPeriodStartDate) &&
         (timeCardDate <= payPeriodEndDate);
}
```

```
double HourlyClassification::
CalculatePayForTimeCard(TimeCard* tc) const
{
    double hours = tc->GetHours();
    double overtime = max(0.0, hours - 8.0);
    double straightTime = hours - overtime;
    return straightTime * itsRate + overtime * itsRate * 1.5;
}
```

Listing 19-46 shows that the `WeeklySchedule` only pays on Fridays.

Listing 19-46

`WeeklySchedule::IsPayDate`

```
bool WeeklySchedule::IsPayDate(const Date& theDate) const
{
  return theDate.GetDayOfWeek() == Date::friday;
}
```

I leave calculating the pay for commissioned employees to you. There shouldn't be any big surprises. As a slightly more interesting exercise, allow time cards to be posted on the weekends, and calculate overtime correctly.

Pay Periods: A Design Problem

Now it's time to implement the union dues and service charges. I'm contemplating a test case that will add a salaried employee, convert it into a union member, and then pay the employee and ensure that the dues were subtracted from his pay. I've coded this in Listing 19-47.

Listing 19-47

`PayrollTest::TestSalariedUnionMemberDues`

```
void PayrollTest::TestSalariedUnionMemberDues()
{
  cerr << "TestSalariedUnionMemberDues" << endl;
  int empId = 1;
  AddSalariedEmployee t(empId, "Bob", "Home", 1000.00);
  t.Execute();
  int memberId = 7734;
  ChangeMemberTransaction cmt(empId, memberId, 9.42);
  cmt.Execute();
  Date payDate(11,30,2001);
  PaydayTransaction pt(payDate);
  pt.Execute();
  ValidatePaycheck(pt, empId, payDate, 1000.0 - ??? );
}
```

Notice the ??? in the last line of the test case. What should I put there? The user stories tell me that union dues are weekly, but salaried employees are paid monthly. How many weeks are in each month? Should I just multiply the dues by four? That's not very accurate. I'll ask the customer what he wants.[5]

The customer tells me that union dues are accrued every Friday. So what I need to do is count the number of Fridays in the pay period and multiply by the weekly dues. There are five Fridays in November, 2001—the month that the test case is written for. So I can modify the test case appropriately.

5. And so Bob talks to himself yet again. Go to `www.google.com/groups` and look up "Schizophrenic Robert Martin."

Counting the Fridays in a pay period implies that I need to know what the starting and ending dates of the pay period are. I have done this calculation before in the function `IsInPayPeriod` in Listing 19-45 (and you probably wrote a similar one for the `CommissionedClassification`). This function is used by the `CalculatePay` function of the `HourlyClassification` object to ensure that only time cards from the pay period are tallied. Now it seems that the `UnionAffiliation` object must call this function too.

But wait! What is this function doing in the `HourlyClassification` class? We've already determined that the association between the payment schedule and the payment classification is accidental. The function that determines the pay period ought to be in the `PaymentSchedule` class, not in the `PaymentClassification` class!

It is interesting that our UML diagrams didn't help us catch this problem. The problem only surfaced when I started thinking about the test cases for `UnionAffiliation`. This is yet another example of how necessary coding feedback is to any design. Diagrams can be useful, but reliance on them without feedback from the code is risky business.

So how do we get the pay period out of the `PaymentSchedule` hierarchy and into the `PaymentClassification` and `Affiliation` hierarchies? These hierarchies do not know anything about each other. We could put the pay period dates into the `Paycheck` object. Right now, the `Paycheck` just has the end date of the pay period. We ought to be able to get the start date in there, too.

Listing 19-48 shows the change made to `PaydayTransaction::Execute()`. Notice that when the `Paycheck` is created, it is passed both the start and end dates of the pay period. If you jump ahead to Listing 19-55 you will see that it is the `PaymentSchedule` that calculates both. The changes to `Paycheck` should be obvious.

Listing 19-48

`PaydayTransaction::Execute()`

```
void PaydayTransaction::Execute()
{
  list<int> empIds;
  GpayrollDatabase.GetAllEmployeeIds(empIds);

  list<int>::iterator i = empIds.begin();
  for (; i != empIds.end(); i++) {
    int empId = *i;
    if (Employee* e = GpayrollDatabase.GetEmployee(empId)) {
      if (e->IsPayDate(itsPayDate)) {
        Paycheck* pc =
          new Paycheck(e->GetPayPeriodStartDate(itsPayDate), itsPayDate);
        itsPaychecks[empId] = pc;
        e->Payday(*pc);
      }
    }
  }
}
```

The two functions in `HourlyClassification` and `CommissionedClassification` that determined if `TimeCards` and `SalesReceipts` were within the pay period have been merged and moved into the base class `PaymentClassification`. (See Listing 19-49.)

Listing 19-49

`PaymentClassification::IsInPayPeriod(...)`

```
bool PaymentClassification::
IsInPayPeriod(const Date& theDate, const Paycheck& pc) const
```

```
{
  Date payPeriodEndDate = pc.GetPayPeriodEndDate();
  Date payPeriodStartDate = pc.GetPayPeriodStartDate();
  return (theDate >= payPeriodStartDate)
      && (theDate <= payPeriodEndDate);
}
```

Now we are ready to calculate the employee's union dues in `UnionAffilliation::CalculateDeductions`. The code in Listing 19-50 shows how this is done. The two dates that define the pay period are extracted from the paycheck and passed to a utility function that counts the number of Fridays between them. This number is then multiplied by the weekly dues rate to calculate the dues for the pay period.

Listing 19-50

`UnionAffiliation::CalculateDeductions()`

```
namespace
{
  int NumberOfFridaysInPayPeriod(const Date& payPeriodStart,
                                 const Date& payPeriodEnd)
  {
    int fridays = 0;
    for (Date day = payPeriodStart; day <= payPeriodEnd; day++)
    {
      if (day.GetDayOfWeek() == Date::friday)
        fridays++;
    }
    return fridays;
  }
}

double UnionAffiliation::
CalculateDeductions(Paycheck& pc) const
{
  double totalDues = 0;

  int fridays =
    NumberOfFridaysInPayPeriod(pc.GetPayPeriodStartDate(),
                               pc.GetPayPeriodEndDate());
  totalDues = itsDues * fridays;
  return totalDues;
}
```

The last two test cases have to do with union service charges. The first test case is shown in Listing 19-51. It makes sure that we deduct service charges appropriately.

Listing 19-51

`PayrollTest::TestHourlyUnionMemberServiceCharge`

```
void PayrollTest::TestHourlyUnionMemberServiceCharge()
{
  cerr << "TestHourlyUnionMemberServiceCharge" << endl;
  int empId = 1;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.24);
  t.Execute();
```

```
  int memberId = 7734;
  ChangeMemberTransaction cmt(empId, memberId, 9.42);
  cmt.Execute();
  Date payDate(11,9,2001);
  ServiceChargeTransaction sct(memberId, payDate, 19.42);
  sct.Execute();
  TimeCardTransaction tct(payDate, 8.0, empId);
  tct.Execute();
  PaydayTransaction pt(payDate);
  pt.Execute();
  Paycheck* pc = pt.GetPaycheck(empId);
  assert(pc);
  assert(pc->GetPayPeriodEndDate() == payDate);
  assertEquals(8*15.24, pc->GetGrossPay(), .001);
  assert("Hold" == pc->GetField("Disposition"));
  assertEquals(9.42 + 19.42, pc->GetDeductions(), .001);
  assertEquals((8*15.24)-(9.42 + 19.42), pc->GetNetPay(), .001);
}
```

The second test case posed something of a problem for me. You can see it in Listing 19-52. This test case makes sure that service charges dated outside the current pay period are not deducted.

Listing 19-52

PayrollTest::TestServiceChargesSpanningMultiplePayPeriods

```
void PayrollTest::
TestServiceChargesSpanningMultiplePayPeriods()
{
  cerr << "TestServiceChargesSpanningMultiplePayPeriods" << endl;
  int empId = 1;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.24);
  t.Execute();
  int memberId = 7734;
  ChangeMemberTransaction cmt(empId, memberId, 9.42);
  cmt.Execute();
  Date earlyDate(11,2,2001); // previous Friday
  Date payDate(11,9,2001);
  Date lateDate(11,16,2001); // next Friday
  ServiceChargeTransaction sct(memberId, payDate, 19.42);
  sct.Execute();
  ServiceChargeTransaction sctEarly(memberId, earlyDate, 100.00);
  sctEarly.Execute();
  ServiceChargeTransaction sctLate(memberId, lateDate, 200.00);
  sctLate.Execute();
  TimeCardTransaction tct(payDate, 8.0, empId);
  tct.Execute();
  PaydayTransaction pt(payDate);
  pt.Execute();
  Paycheck* pc = pt.GetPaycheck(empId);
  assert(pc);
  assert(pc->GetPayPeriodEndDate() == payDate);
```

```
  assertEquals(8*15.24, pc->GetGrossPay(), .001);
  assert("Hold" == pc->GetField("Disposition"));
  assertEquals(9.42 + 19.42, pc->GetDeductions(), .001);
  assertEquals((8*15.24)-(9.42 + 19.42), pc->GetNetPay(), .001);
}
```

To implement this, I wanted `UnionAffiliation::CalculateDeductions` to call `IsInPayPeriod`. Unfortunately, we just put `IsInPayPeriod` in the `PaymentClassification` class. (See Listing 19-49.) It was convenient to put it there while it was the derivatives of `PaymentClassification` that needed to call it. But now other classes need it as well. So I moved the function into the `Date` class. After all, the function is simply determining if a given date is between two other given dates. (See Listing 19-53.)

Listing 19-53

`Date::IsBetween`

```
static bool IsBetween(const Date& theDate,
                      const Date& startDate,
                      const Date& endDate)
{
  return (theDate >= startDate) && (theDate <= endDate);
}
```

Now, we can finally finish the `UnionAffiliation::CalculateDeductions` function. I leave that as an exercise for you.

Listings 19-54 and 19-55 show the implementation of the `Employee` class.

Listing 19-54

`Employee.h`

```
#ifndef EMPLOYEE_H
#define EMPLOYEE_H

#include <string>

class PaymentSchedule;
class PaymentClassification;
class PaymentMethod;
class Affiliation;
class Paycheck;
class Date;

class Employee
{
 public:
  virtual ~Employee();
  Employee(int empid, string name, string address);
  void SetName(string name);
  void SetAddress(string address);
  void SetClassification(PaymentClassification*);
  void SetMethod(PaymentMethod*);
  void SetSchedule(PaymentSchedule*);
  void SetAffiliation(Affiliation*);
```